CORE FUNCTIONALITY

Week 2 Implementation Report

Project: Historical Manuscript Fragment Reconstruction & Evidence Explorer

1. Introduction

This document describes the main functions completed for the Week 2 milestone. The first version of the application allows a user to add manuscript page information, upload an image, view saved records, and search for a manuscript fragment. These functions provide the starting point for the later text and image comparison work.

Five requirements from the original functional requirements list were selected: FR-13, FR-14, FR-15, FR-19, and FR-20.

2. Selected Requirements and Implemented Functionality

FR-13 – Upload manuscript page images

A user can choose a manuscript image and upload it through the Add Manuscript Fragment form. The application accepts PNG, JPG, JPEG, TIF, TIFF, and WEBP filename extensions. The uploaded image is saved in the local uploads folder. A unique filename is created to reduce the chance of overwriting another uploaded file.

> **Source files:** app/fragments.py – new_fragment() and allowed_file(); app/templates/fragments/new.html – image upload form; app/__init__.py – creates the upload folder.

FR-14 – Add manuscript details

A user can enter the manuscript title, fragment identifier, and description. The title and fragment identifier are required. The application saves these details in the SQLite database together with the image filename.

> **Source files:** app/fragments.py – new_fragment(); app/database.py – manuscript_fragments table; app/templates/fragments/new.html – manuscript details form.

FR-15 – Enter page information

A user can enter a page number and manuscript reference. These fields are saved with the manuscript record. They can be left blank when the information is not available.

> **Source files:** app/fragments.py – new_fragment(); app/database.py – manuscript_fragments table; app/templates/fragments/new.html – page number and reference fields.

FR-19 – View manuscript fragments

A user can view the saved manuscript fragments on the dashboard. The user can open a fragment to see its image, title, identifier, page number, reference, description, transcription, and date added.

Source files: app/fragments.py – index() and fragment_detail(); app/templates/fragments/index.html – fragment list; app/templates/fragments/detail.html – fragment details; app/templates/base.html – shared page layout.

FR-20 – Search and filter manuscript fragments

A user can search saved fragments by title, fragment identifier, or page number. The application uses the search text to find matching records in SQLite and displays the results on the dashboard. A Clear link returns the user to the full list.

Source files: app/fragments.py – index(); app/templates/fragments/index.html – search form.

3. Source Modules and Components

The application is split into small files so that each part has a clear purpose.

File or folder	Purpose
run.py	Starts the Flask application.
config.py	Stores application settings, upload size limit, and allowed image extensions.
app/__init__.py	Creates the Flask application, upload folder, database connection setup, and routes.
app/database.py	Connects to SQLite, creates the manuscript_fragments table, and closes database connections.
app/fragments.py	Contains the routes and logic for adding, listing, viewing, searching, and serving uploaded images.
app/templates/	Contains the HTML pages used by the application.
app/static/css/style.css	Controls the appearance and layout of the pages.
tests/	Contains automated tests for the application routes and fragment operations.
requirements.txt	Lists Flask and pytest as project dependencies.

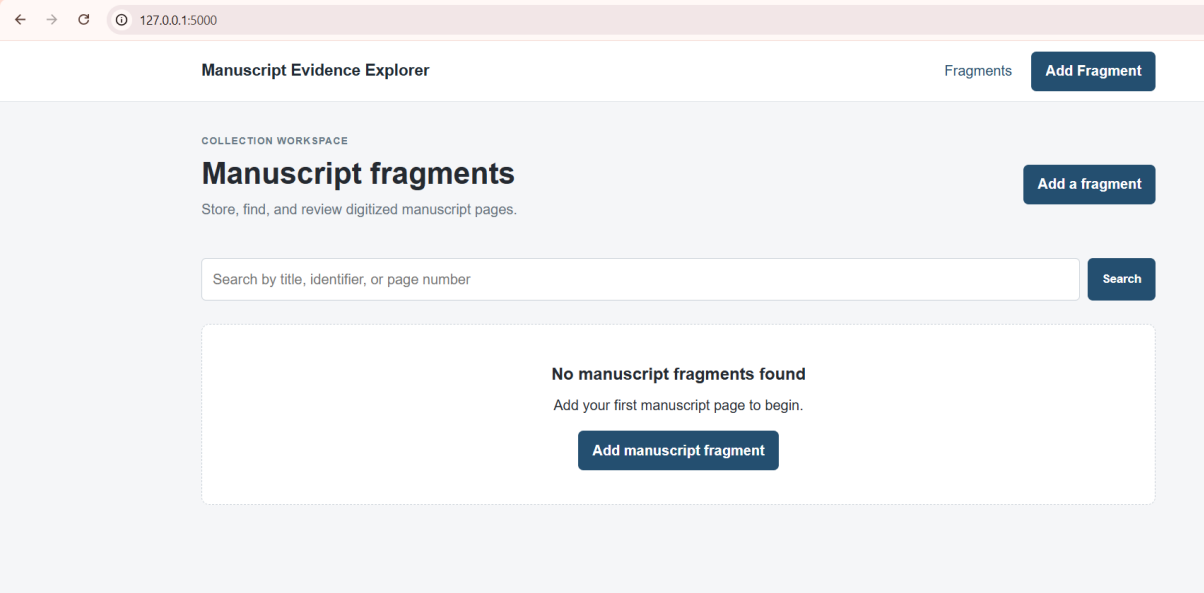
4. Tests Performed and Results

Manual checks were completed in the browser using a sample manuscript record. The screenshots from the running application showed that the upload, save, detail view, and search workflow operated as expected.

Requirement	Manual test	Observed result
FR-13	Uploaded a manuscript page image through the form.	Successful – image appeared on the dashboard and detail page.

FR-14	Entered a title, identifier, and description and saved the record.	Successful – the dashboard showed the saved title and identifier.
FR-15	Entered page number 1 and a manuscript reference.	Successful – both values appeared on the detail page.
FR-19	Opened the saved fragment from the dashboard.	Successful – image and stored details were displayed.
FR-20	Searched for MS-001.	Successful – the matching fragment appeared in the results.

Automated tests are included in tests/test_fragments.py. They cover the dashboard, saving an uploaded fragment, missing required information, unsupported file extensions, the detail page, and search by identifier. The automated test command is 'python -m pytest -v'. The automated test results have not yet been provided or confirmed, so this report does not claim that the automated tests passed.



127.0.0.1:5000/fragments/new

Manuscript Evidence Explorer

Fragments | Add Fragment

FRAGMENT MANAGEMENT

Add manuscript fragment

Enter the available information. Unknown details can be left blank.

Manuscript title *

Medieval English Historical Manuscript

Fragment identifier *

MS-001

Page number

1

Manuscript reference

Historical Collection 001

Description

An old handwritten English manuscript page containing historical text.

Transcribed text

This manuscript contains historical information about the traditions, culture, and events of an earlier period.

Manuscript page image *

Choose File | example.jpeg

Supported: PNG, JPG, JPEG, TIF, TIFF, WEBP. Maximum size: 10 MB.

Save fragment | Cancel

127.0.0.1:5000

Manuscript Evidence Explorer

Fragments | Add Fragment

Manuscript fragment saved successfully.

COLLECTION WORKSPACE

Manuscript fragments

Store, find, and review digitized manuscript pages.

Add a fragment

Search by title, identifier, or page number

Search



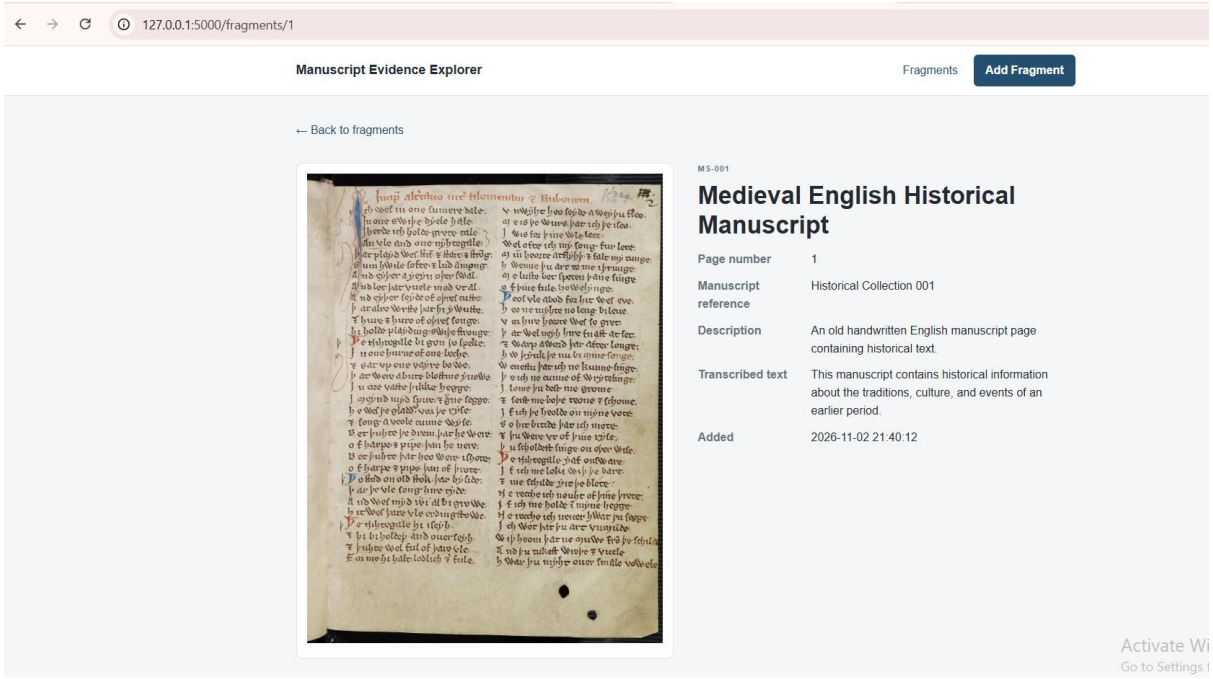
MS-001

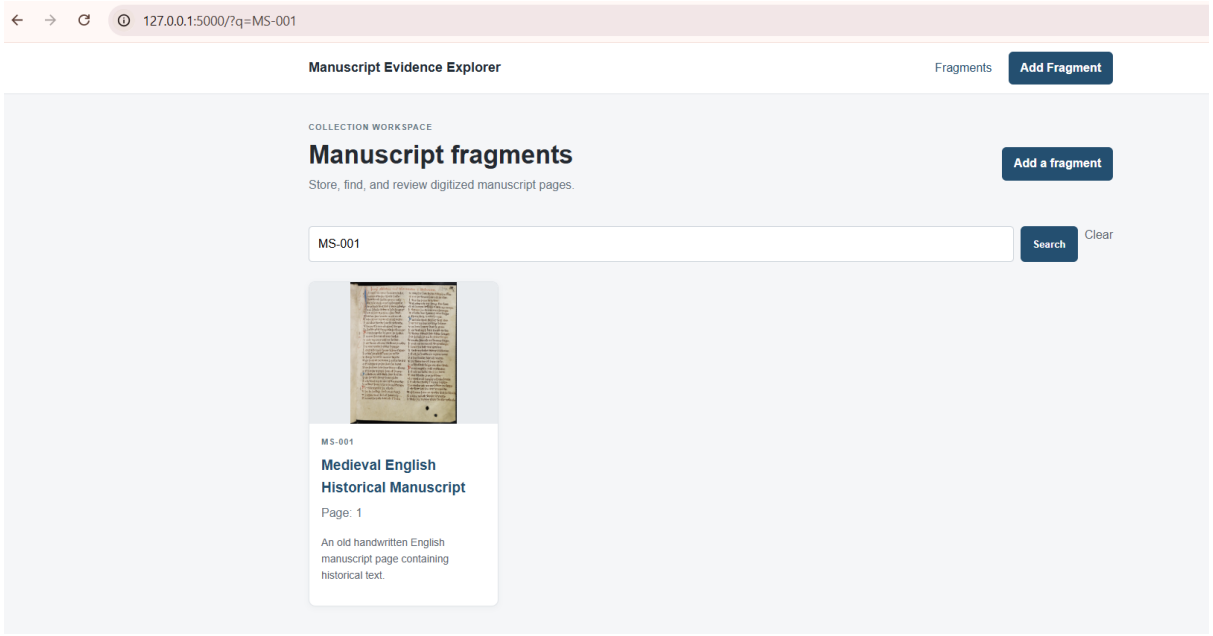
Medieval English Historical Manuscript

Page: 1

An old handwritten English manuscript page containing historical text.

Activate





5. Technical Problems Encountered

No major application error was reported during the manual checks shown so far. The Flask application started in the local development environment, and the sample fragment was saved and displayed. Automated testing still needs to be run in the user's VS Code environment after installing the dependencies.

One current limitation is that image type validation checks the file extension. It does not yet inspect the actual file content to confirm that the file is a valid image. This should be improved before the application is used with important collections.

6. Incomplete Functionality and Further Work

- Automated tests must be run with 'python -m pytest -v' and the actual results recorded.
- User registration, login, logout, and role-based access are not implemented yet.
- Editing and deleting manuscript records are not implemented yet.
- Image validation should check the actual image content, not only the filename extension.
- Text similarity comparison has not been implemented.
- Image feature extraction and image similarity comparison have not been implemented.
- Graph-based fragment relationships using NetworkX have not been implemented.
- Candidate page ordering and constraints using OR-Tools have not been implemented.
- Evidence scoring, ranking, and comparison result history are future work.

7. Summary

The Week 2 version provides a working starting point for manuscript fragment management. A user can upload an image, enter manuscript details, save the record, view it, and search for it. The five selected requirements were checked manually in the browser. The next step is to run the automated tests, record their actual results, and continue with the comparison and reconstruction features in later development work.